

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Nguyễn Xuân Việt Đức - 22280012

Bài tập thực hành - Tuần 2

Bài toán:

Có 3 người truyền giáo và 3 con quỷ ở bờ bên trái của một con sông, cùng với con thuyền có thể chở được 1 hoặc 2 người. Nếu số quỷ nhiều hơn số người truyền giáo trong một bờ thì số quỷ sẽ ăn thịt số người truyền giáo. Tìm các để đưa tất cả qua bờ sông bên kia (bên phải) sao cho số người không ít hơn số quỷ ở cùng 1 bờ (bên trái hay bên phải), nghĩa là không ai bị ăn thịt. Gọi (a, b, k) với $0 \leq a, b \leq 3$, trong đó a là số người, b là số con quỷ ở bên bờ bên trái, $k = 1$ nếu thuyền ở bờ bên trái và $k = 0$ nếu thuyền ở bờ bên phải. Khi đó, không gian trạng thái của bài toán được xác định như sau:

- Trạng thái ban đầu là $(3, 3, 1)$
- Thuyền chở qua sông 1 người, hoặc 1 con quỷ, hoặc 1 người và 1 con quỷ, hoặc 2 người, hoặc 2 con quỷ $\Rightarrow$ các phép toán chuyển từ trạng thái này sang trạng thái khác là: $(1, 0), (0, 1), (1, 1), (2, 0), (0, 2)$ (trong đó (x, y) là số người và số quỷ di chuyển từ bờ bên trái qua bờ bên phải hay ngược lại).
- Trạng thái kết thúc là $(0, 0, 0)$.

Đã cài đặt và thực thi chương trình

Báo cáo:

generate_full_space_tree.py:

```
def is_valid_move(number_missionaries, number_cannibals):  
    """  
    Checks if number  
    """  
    return (0 <= number_missionaries <= 3) and (0 <= number_cannibals <= 3)
```

- Chức năng: Kiểm tra xem trạng thái có hợp lệ hay không.

- Hoạt động: Hàm nhận số lượng người truyền giáo (number_missionaries) và số lượng thổ dân ăn thịt (number_cannibals) rồi kiểm tra xem các giá trị này có nằm trong phạm vi cho phép (từ 0 đến 3) hay không.
- Trả về: True nếu trạng thái hợp lệ và False nếu không.

```
def write_image(file_name="state_space"):
    try:
        graph.write_png(f"{file_name}_{max_depth}.png")
    except Exception as e:
        print("Error while writting file", e)
    VietDucFCB
    print(f"File {file_name}_{max_depth}.png successfully written.")
```

- Chức năng: Ghi hình ảnh của đồ thị trạng thái vào một file .png.
- Hoạt động: Hàm gọi phương thức write_png của graph để tạo ra một file hình ảnh với tên chứa độ sâu tối đa (max_depth). Nếu có lỗi khi ghi file, hàm sẽ in thông báo lỗi.
- Trả về: Không trả về giá trị gì nhưng in thông báo nếu ghi file thành công hoặc lỗi.

```
def draw_edge(number_missionaries, number_cannibals, side, depth_level, node_num):
    u, v = None, None
    if Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)] is not None:
        u = pydot.Node(str(Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)]),
            label=str(Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)][:-3]))
        graph.add_node(u)
        v = pydot.Node(str((number_missionaries, number_cannibals, side, depth_level, node_num)),
            label=str((number_missionaries, number_cannibals, side)))
        graph.add_node(v)

        edge = pydot.Edge(str(Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)]),
            str((number_missionaries, number_cannibals, side, depth_level, node_num)), dir='forward')
        graph.add_edge(edge)
    else:
        #For start node
        v = pydot.Node(str((number_missionaries, number_cannibals, side, depth_level, node_num)),
            label=str((number_missionaries, number_cannibals, side)))
        graph.add_node(v)
    VietDucFCB
    return u,v
```

- Chức năng: Tạo các nút (node) và cạnh (edge) trong đồ thị trạng thái.
- Hoạt động: Hàm kiểm tra trạng thái của nút hiện tại trong Parent. Nếu có cha (Parent không phải là None), hàm sẽ tạo nút u cho cha và nút v cho nút hiện tại, sau đó tạo một cạnh từ u đến v. Nếu không có cha (điểm bắt đầu), chỉ tạo nút v.

- Trả về: Trả về hai đối tượng u và v, trong đó u là nút cha (có thể là None nếu không có cha), và v là nút hiện tại.

```
def is_start_state(number_missionaries, number_cannibals, side):
    return (number_missionaries, number_cannibals, side) == (3, 3, 1)
```

- Chức năng: Kiểm tra xem trạng thái hiện tại có phải là trạng thái bắt đầu hay không.
- Hoạt động: Hàm so sánh trạng thái hiện tại (number_missionaries, number_cannibals, side) với trạng thái bắt đầu (3, 3, 1).
- Trả về: True nếu là trạng thái bắt đầu, False nếu không phải.

```
def is_goal_state(number_missionaries, number_cannibals, side):
    return (number_missionaries, number_cannibals, side) == (0, 0, 0)
```

- Chức năng: Kiểm tra xem trạng thái hiện tại có phải là trạng thái mục tiêu hay không.
- Hoạt động: Hàm so sánh trạng thái hiện tại với trạng thái mục tiêu (0, 0, 0).
- Trả về: True nếu là trạng thái mục tiêu, False nếu không phải.

```
def number_of_cannibals_exceeds(number_missionaries, number_cannibals):
    number_missionaries_right = 3 - number_missionaries
    number_cannibals_right = 3 - number_cannibals
    return (number_missionaries > 0 and number_cannibals > number_missionaries) \
        or (number_missionaries_right > 0 and number_cannibals_right > number_missionaries_right)
```

- Chức năng: Kiểm tra nếu số lượng thổ dân ăn thịt vượt quá số người truyền giáo ở bất kỳ bên nào của sông.
- Hoạt động: Hàm tính toán số lượng người truyền giáo và thổ dân ở mỗi bên sông, rồi kiểm tra nếu số thổ dân nhiều hơn số người truyền giáo ở bất kỳ bên nào (vì điều này là điều kiện không hợp lệ trong bài toán).
- Trả về: True nếu trạng thái không hợp lệ do số lượng thổ dân vượt quá, False nếu trạng thái hợp lệ.

```

def generate():
    global i
    q = deque()
    node_num = 0
    q.append((3, 3, 1, 0, node_num))

    Parent[(3, 3, 1, 0, node_num)] = None

    while q:

        number_missionaries, number_cannibals, side, depth_level, node_num = q.popleft()
        # print ( number_missionaries, number_cannibals )
        # Draw Edge from u -> v
        # Where u = Parent[v]
        # and v= (number_missionaries, number_cannibals, side, depth_level)
        u, v = draw_edge(number_missionaries, number_cannibals, side, depth_level, node_num)

        if is_start_state(number_missionaries, number_cannibals, side):
            v.set_style("filled")
            v.set_fillcolor("blue")
            v.set_fontcolor("white")
        elif is_goal_state(number_missionaries, number_cannibals, side):
            v.set_style("filled")
            v.set_fillcolor("green")
            continue
            # return True

        elif number_of_cannibals_exceeds(number_missionaries, number_cannibals):
            v.set_style("filled")
            v.set_fillcolor("red")
            continue
        else:
            v.set_style("filled")
            v.set_fillcolor("orange")

        if depth_level == max_depth:
            return True

        op = -1 if side == 1 else 1

        can_be_expanded = False

        # i = node_num
        for x, y in options:
            next_m, next_c, next_s = number_missionaries + op * x, number_cannibals + op * y, int(not side)

            if Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)] is None or (
                next_m, next_c, next_s) \
                != Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)][1:3]:
                if is_valid_move(next_m, next_c):
                    can_be_expanded = True
                    i += 1
                    q.append((next_m, next_c, next_s, depth_level + 1, i))
                    # Keep track of parent
                    Parent[(next_m, next_c, next_s, depth_level + 1, i)] = \
                        (number_missionaries, number_cannibals, side, depth_level, node_num)
            if not can_be_expanded:
                v.set_style("filled")
                v.set_fillcolor("gray")
    return False

```

- Chức năng: Tạo và duyệt qua không gian trạng thái của bài toán, sử dụng deque để triển khai duyệt rộng (BFS).
- Hoạt động: Hàm bắt đầu với trạng thái khởi đầu (3, 3, 1) và thêm nó vào hàng đợi q. Trong khi duyệt qua các trạng thái trong q, nó:
 - Sử dụng hàm draw_edge để vẽ cạnh từ nút cha đến nút hiện tại.
 - Kiểm tra xem trạng thái hiện tại có phải là trạng thái bắt đầu, trạng thái mục tiêu, hoặc trạng thái không hợp lệ không. Các trạng thái này được tô màu tương ứng.
 - Nếu độ sâu vượt quá max_depth, kết thúc hàm.
 - Với mỗi trạng thái hợp lệ, hàm tạo ra các trạng thái con dựa trên options và thêm chúng vào hàng đợi q nếu chúng chưa được duyệt.
- Trả về: True nếu đã duyệt đến độ sâu tối đa, False nếu không.

```
if __name__ == "__main__":
    if generate():
        write_image()
```

- Chức năng: Chạy hàm generate() và gọi write_image() nếu đồ thị đã được tạo.
- Hoạt động: Đầu tiên, generate() được gọi để xây dựng đồ thị không gian trạng thái. Nếu thành công, write() sẽ ghi đồ thị vào file .png.

solve.py

```
def __init__(self):
    # Start state (3M, 3C, Left)
    # Goal state (0M, 0C, Right)
    # Each state gives the number of missionaries and cannibals on the left side

    self.start_state = (3, 3, 1)
    self.goal_state = (0, 0, 0)
    self.options = [(1, 0), (0, 1), (1, 1), (0, 2), (2, 0)]

    self.boat_side = ["right", "left"]

    self.graph = pydot.Dot(graph_type='graph', bgcolor="#fff3af",
                           label="fig: Missionaries and Cannibals State Space Tree", fontcolor="red", fontsize="24")
    self.visited = {}
    self.solved = False
```

- Chức năng: Khởi tạo đối tượng cho lớp Solution.
- Hoạt động:
 - Thiết lập trạng thái bắt đầu là (3, 3, 1) (3 nhà truyền giáo và 3 kẻ ăn thịt ở phía bên trái).
 - Thiết lập trạng thái đích là (0, 0, 0) (cả 3 nhà truyền giáo và kẻ ăn thịt đã được chuyển sang phía bên phải).
 - Khai báo các phép di chuyển có thể thực hiện được với danh sách self.options.
 - Thiết lập biểu đồ và các biến theo dõi.

```
def is_valid_move(self, number_missionaries, number_cannibals):
    """
    Check if number constraints are statisfied
    """
    return (0 <= number_missionaries <= 3) and (0 <= number_cannibals <= 3)

def is_start_state(self, number_missionaries, number_cannibals, side):
    return (number_missionaries, number_cannibals, side) == self.start_state

def is_start_state(self, number_missionaries, number_cannibals, side):
    return (number_missionaries, number_cannibals, side) == self.start_state

def number_of_cannibals_exceeds(self, number_missionaries, number_cannibals):
    number_missionaries_right = 3 - number_missionaries
    number_cannibals_right = 3 - number_cannibals
    return (number_missionaries > 0 and number_cannibals > number_missionaries) \
        or (number_missionaries_right > 0 and number_cannibals_right > number_missionaries_right)
```

```
def write_image(self, file_name="state_space.png"):
    try:
        self.graph.write_png(file_name)
    except Exception as e:
        print("Error writing file ", e)
    print(f"File {file_name} written successfully")
```

Các hàm trên có chức năng, hoạt động và trả về tương tự như các hàm cùng tên trong file generate_full_space_tree.py

```
def solve(self, solve_method="dfs"):
    self.visited = dict()
    Parent[self.start_state] = None
    Move[self.start_state] = None
    node_list[self.start_state] = None

    return self.dfs(*self.start_state, number_missionaries: 0) if solve_method == "dfs" else self.bfs()
```

- Chức năng: Chọn phương pháp giải thuật và khởi chạy quá trình giải bài toán.
- Hoạt động: Thiết lập biến khởi tạo và tùy thuộc vào phương pháp (dfs hoặc bfs), gọi hàm tương ứng để giải bài toán.
- Trả về: Kết quả của phương pháp tìm kiếm được chọn.

```
def draw_legend(self):
    """
    Utility method to draw legend on graph if legend flag is ON
    """
    graphlegend = pydot.Cluster(graph_name="Legend", label="Legend", fontsize="20", color="gold",
                                fontcolor="blue", style="filled", fillcolor="#f4f4f4")

    node1 = pydot.Node(name="1", style="filled", fillcolor="blue", label="Start Node", fontcolor="white", width="2",
                        fixedsize="true")
    graphlegend.add_node(node1)

    node2 = pydot.Node(name="2", style="filled", fillcolor="red", label="Killed Node", fontcolor="black", width="2",
                        fixedsize="true")
    graphlegend.add_node(node2)

    node3 = pydot.Node(name="3", style="filled", fillcolor="yellow", label="Solution Node", width="2", fixedsize="true")
    graphlegend.add_node(node3)

    node4 = pydot.Node(name="4", style="filled", fillcolor="gray", label="Can't be expanded", width="2",
                        fixedsize="true")
    graphlegend.add_node(node4)

    node5 = pydot.Node(name="5", style="filled", fillcolor="green", label="Goal Node", width="2", fixedsize="true")
    graphlegend.add_node(node5)
```

```

node7 = pydot.Node( name="7", style="filled", fillcolor="gold", label="Node with child", width="2", fixedsize="true")
graphlegend.add_node(node7)

description = "Each node (m, c, s) represents a \nstate where 'm' is the number of\n missionaries,\ 'c' is the cannibals,\ 's' is the side of the boat\n"
" Where 'l' represents the left \nside and '0' the right side \n\nOur objective is to reach goal state (0, 0, 0)\nfrom start state (3, 3, 1) by some \noperations = [(1, 0), (0, 1), (2, 0), (0, 2), (1, 1)]\n" \
"Each tuple (x, y) inside operators \nrepresents the number of missionaries and\ncannibals to be moved from left to right \nif s == 1 and vice versa."

node6 = pydot.Node( name="6", style="filled", fillcolor="gold", label=description, shape="plaintext", fontsize=20, fontcolor="red")
graphlegend.add_node(node6)

self.graph.add_subgraph(graphlegend)

self.graph.add_edge(pydot.Edge(node1, node2, style="invis"))
self.graph.add_edge(pydot.Edge(node2, node3, style="invis"))
self.graph.add_edge(pydot.Edge(node3, node4, style="invis"))
self.graph.add_edge(pydot.Edge(node4, node5, style="invis"))
self.graph.add_edge(pydot.Edge(node5, node7, style="invis"))
self.graph.add_edge(pydot.Edge(node7, node6, style="invis"))

```

- Chức năng: Tạo và hiển thị chú thích cho biểu đồ trạng thái.
- Hoạt động: Tạo các nút chú thích mô tả ý nghĩa của các loại trạng thái khác nhau và thêm vào biểu đồ graphlegend.
Các nút đại diện cho các trạng thái khác nhau, mỗi nút có màu và nhãn riêng để mô tả ý nghĩa của nó:

- node1: Đại diện cho trạng thái bắt đầu (start node) với màu xanh (blue) và nhãn "Start Node".
- node2: Đại diện cho trạng thái bị loại (killed node) khi các điều kiện vi phạm luật trò chơi. Nút này có màu đỏ (red) và nhãn "Killed Node".
- node3 với màu vàng (yellow) cho nút trong giải pháp (Solution Node).
- node4 với màu xám (gray) cho nút không thể mở rộng (Can't be expanded).
- node5 với màu xanh lá (green) cho trạng thái đích (goal node).
- node7 với màu vàng (gold) cho nút có các nút con (Node with child).

Thêm các nút và mô tả vào cụm chú giải, các cạnh ẩn để sắp xếp các nút.

```
def draw(self, *, number_missionaries_left, number_cannibals_left, number_missionaries_right,
        number_cannibals_right):
    """
        raw state on console using emojis
    """
    left_m = emoji.emojize(f"old_man: " * number_missionaries_left)
    left_c = emoji.emojize(f"ogre: " * number_cannibals_left)
    right_m = emoji.emojize(f":old_man: " * number_missionaries_right)
    right_c = emoji.emojize(f":ogre: " * number_cannibals_right)

    print('{}{}{}{}{}'.format(*args: left_m, left_c + " " * (14 - len(left_m) - len(left_c)), _\
                                " " * 40, " " * (12 - len(right_m) - len(right_c)) + right_m, right_c))
    print("")
```

- Chức năng: Hiển thị trạng thái hiện tại của hai bờ sông.
- Hoạt động: Dùng emoji để biểu diễn nhà truyền giáo và kẻ ăn thịt ở mỗi bờ, sắp xếp khoảng cách để hiển thị đẹp.
 - Khởi tạo một đối tượng đồ thị sử dụng thư viện pydot.
 - Thêm các nút (nodes) đại diện cho từng trạng thái của trò chơi, sử dụng màu sắc và nhãn để phân biệt các trạng thái khác nhau.
 - Thêm các cạnh (edges) giữa các nút để thể hiện các mối quan hệ và hành động có thể xảy ra trong trò chơi.
 - Gọi hàm draw_legend để thêm chú giải (legend) vào biểu đồ, giúp người dùng hiểu rõ hơn về ý nghĩa của các nút.

```
def show_solution(self):
    # Recursively start from Goal State
    # And find parent until start state is reached

    state = self.goal_state
    path, steps, nodes = [], [], []

    while state is not None:
        path.append(state)
        steps.append(Move[state])
        nodes.append(node_list[state])

        state = Parent[state]

    steps, nodes = steps[::-1], nodes[::-1]

    number_missionaries_left, number_cannibals_left = 3, 3
    number_missionaries_right, number_cannibals_right = 0, 0

    print("*" * 60)
    self.draw(number_missionaries_left=number_missionaries_left, number_cannibals_left=number_cannibals_left,
              number_missionaries_right=number_missionaries_right, number_cannibals_right=number_cannibals_right)
```

```

for i, ((number_missionaries, number_cannibals, side), node) in enumerate(zip(steps[1:], nodes[1:])):

    if node.get_label() != str(self.start_state):
        node.set_style("filled")
        node.set_fillcolor("yellow")

    print(f"Step {i + 1}: Move {number_missionaries} missionaries and {number_cannibals} \
        cannibals from {self.boat_side[side]} to {self.boat_side[int(not side)]}")

    op = -1 if side == 1 else 1

    number_missionaries_left = number_missionaries_left + op * number_missionaries
    number_cannibals_left = number_cannibals_left + op * number_cannibals

    number_missionaries_right = number_missionaries_right - op * number_missionaries
    number_cannibals_right = number_cannibals_right - op * number_cannibals

    self.draw(number_missionaries_left=number_missionaries_left, number_cannibals_left=number_cannibals_left,
              number_missionaries_right=number_missionaries_right,
              number_cannibals_right=number_cannibals_right)

    print("Congratulations!!! You have solve the problem")
    print("a" * 60)

```

- Chức năng: Hàm này hiển thị giải pháp cho trò chơi bằng cách làm nổi bật các trạng thái (nodes) mà trò chơi đã đi qua để đạt được trạng thái đích.
- Hoạt động:
 - Nhận vào danh sách các nút đại diện cho giải pháp (solution path).
 - Duyệt qua danh sách các nút trong giải pháp, thay đổi màu sắc của các nút để làm nổi bật chúng trong đồ thị.
 - Gọi hàm draw để cập nhật và hiển thị lại đồ thị sau khi đã thay đổi màu sắc của các nút.

```

def draw_edge(self, number_missionaries, number_cannibals, side, depth_level):
    u, v = None, None
    if Parent[(number_missionaries, number_cannibals, side)] is not None:
        u = pydot.Node(str(Parent[(number_missionaries, number_cannibals, side)] + (depth_level - 1,)),
                       label=str(Parent[(number_missionaries, number_cannibals, side)]))
        self.graph.add_node(u)

        v = pydot.Node(str((number_missionaries, number_cannibals, side, depth_level)),
                       label=str((number_missionaries, number_cannibals, side)))
        self.graph.add_node(v)

        edge = pydot.Edge(str(Parent[(number_missionaries, number_cannibals, side)] + (depth_level - 1,)),
                          str((number_missionaries, number_cannibals, side, depth_level)), dir="forward")
        self.graph.add_edge(edge)
    else:
        # For start node
        v = pydot.Node(str((number_missionaries, number_cannibals, side, depth_level)),
                       label=str((number_missionaries, number_cannibals, side)))
        self.graph.add_node(v)

    return u, v

```

- Chức năng: Hàm này vẽ các cạnh (edges) giữa hai nút trong đồ thị, thể hiện các hành động chuyển đổi trạng thái trong trò chơi.
- Hoạt động:
 - Nhận vào hai nút và một nhãn cho cạnh.
 - Tạo một cạnh với màu sắc cụ thể, tùy thuộc vào hành động được thực hiện (ví dụ: di chuyển 1 hoặc 2 người).
 - Thêm cạnh này vào đồ thị để thể hiện mối liên kết giữa các trạng thái.

```
def bfs(self):
    q = deque()
    q.append(self.start_state + (0,))
    self.visited[self.start_state] = True

    while q:
        number_missionaries, number_cannibals, side, depth_level = q.popleft()

        # Draw Edge from u -> v
        # Where u = Parent[v]
        # and v = (number_missionaries, number_cannibals, side, depth_level)
        u, v = self.draw_edge(number_missionaries, number_cannibals, side, depth_level)

        if self.is_start_state(number_missionaries, number_cannibals, side):
            v.set_style("filled")
            v.set_fillcolor("blue")
            v.set_fontcolor("white")
        elif self.is_goal_state(number_missionaries, number_cannibals, side):
            v.set_style("filled")
            v.set_fillcolor("green")
            return True
        elif self.number_of_cannibals_exceeds(number_missionaries, number_cannibals):
            v.set_style("filled")
            v.set_fillcolor("red")
            continue
```

```

else:
    v.set_style("filled")
    v.set_fillcolor("orange")

op = -1 if side == 1 else 1
can_be_expanded = False

for x, y in self.options:
    next_m = number_missionaries + op * x
    next_c = number_cannibals + op * y
    next_s = int(not side)

    if (next_m, next_c, next_s) not in self.visited:
        if self.is_valid_move(next_m, next_c):
            can_be_expanded = True
            self.visited[(next_m, next_c, next_s)] = True
            q.append((next_m, next_c, next_s, depth_level + 1))

            # Keep track of parent and corresponding move
            Parent[(next_m, next_c, next_s)] = (number_missionaries, number_cannibals, side)
            Move[(next_m, next_c, next_s)] = (x, y, side)
            node_list[(next_m, next_c, next_s)] = v

if not can_be_expanded:
    v.set_style("filled")
    v.set_fillcolor("gray")

return False

```

- Chức năng: Tìm kiếm trạng thái đích trong không gian trạng thái bằng phương pháp tìm kiếm theo chiều rộng.
- Hoạt động
 - Tạo một hàng đợi (queue) và một tập hợp (set) để theo dõi các trạng thái đã khám phá.
 - Thêm trạng thái ban đầu vào hàng đợi và đánh dấu nó là đã khám phá.
 - Lặp lại quá trình cho đến khi hàng đợi rỗng:
 - * Lấy trạng thái ở đầu hàng đợi.
 - * Kiểm tra xem trạng thái này có phải là trạng thái đích không.
 - * Nếu là trạng thái đích, lưu lại đường đi và trả về trạng thái này.
 - * Nếu không, sinh ra tất cả các trạng thái con khả thi từ trạng thái hiện tại.
 - * Đối với mỗi trạng thái con: Nếu chưa được khám phá, thêm vào hàng đợi và đánh dấu là đã khám phá.
 - Nếu hàng đợi rỗng mà vẫn chưa tìm thấy trạng thái đích, kết thúc quá trình và trả về thông báo rằng không có giải pháp.

- Trả về:
 - Trả về một đường đi (có thể là danh sách các trạng thái từ trạng thái ban đầu đến trạng thái đích) nếu tìm thấy.
 - Nếu không tìm thấy, có thể trả về None hoặc một thông báo rằng không có giải pháp.

```
def dfs(self, number_missionaries, number_cannibals, side, depth_level):
    self.visited[(number_missionaries, number_cannibals, side)] = True

    # Draw Edge (from u -> v)
    # Where u = Parent[v]
    u, v = self.draw_edge(number_missionaries, number_cannibals, side, depth_level)

    if self.is_start_state(number_missionaries, number_cannibals, side):
        v.set_style("filled")
        v.set_fillcolor("blue")
    elif self.is_goal_state(number_missionaries, number_cannibals, side):
        v.set_style("filled")
        v.set_fillcolor("green")
        return True
    elif self.number_of_cannibals_exceeds(number_missionaries, number_cannibals):
        v.set_style("filled")
        v.set_fillcolor("red")
        return False
    else:
        v.set_style("filled")
        v.set_fillcolor("orange")

    solution_found = False
    operation = -1 if side == 1 else 1

    can_be_expanded = False
```

```

for x, y in self.options:
    next_m, next_c, next_s = number_missionaries + operation * x, number_cannibals + operation * y, int(
        not side)

    if (next_m, next_c, next_s) not in self.visited:
        if self.is_valid_move(next_m, next_c):
            can_be_expanded = True
            # Keep track of Parent state and corresponding move
            Parent[(next_m, next_c, next_s)] = (number_missionaries, number_cannibals, side)
            Move[(next_m, next_c, next_s)] = (x, y, side)
            node_list[(next_m, next_c, next_s)] = v

            solution_found = (solution_found or self.dfs(next_m, next_c, next_s, depth_level + 1))

            if solution_found:
                return True

if not can_be_expanded:
    v.set_style("filled")
    v.set_fillcolor("gray")

self.solved = solution_found
return solution_found

```

- Chức năng: Tìm kiếm trạng thái đích trong không gian trạng thái bằng phương pháp tìm kiếm theo chiều sâu
- Hoạt động:
 - Tạo một ngăn xếp (stack) và một tập hợp (set) để theo dõi các trạng thái đã khám phá.
 - Thêm trạng thái ban đầu vào ngăn xếp và đánh dấu nó là đã khám phá.
 - Lặp lại quá trình cho đến khi ngăn xếp rỗng:
 - * Lấy trạng thái ở đỉnh ngăn xếp.
 - * Kiểm tra xem trạng thái này có phải là trạng thái đích không.
 - * Nếu là trạng thái đích, lưu lại đường đi và trả về trạng thái này.
 - * Nếu không, sinh ra tất cả các trạng thái con khả thi từ trạng thái hiện tại.
 - Đối với mỗi trạng thái con: Nếu chưa được khám phá, thêm vào ngăn xếp và đánh dấu là đã khám phá.
 - Nếu ngăn xếp rỗng mà vẫn chưa tìm thấy trạng thái đích, kết thúc quá trình và trả về thông báo rằng không có giải pháp.

- Trả về:
 - Trả về một đường đi (có thể là danh sách các trạng thái từ trạng thái ban đầu đến trạng thái đích) nếu tìm thấy.
 - Nếu không tìm thấy, có thể trả về None hoặc một thông báo rằng không có giải pháp.

main.py:

```
def main():
    s = Solution()

    if s.solve(solve_method):
        # Display Solution on console
        s.show_solution()

        output_file_name = f"{solve_method}"
        # Draw legend if legend_flag is set
        if legend_flag:
            if legend_flag[0].upper() == 'T':
                output_file_name += "_legend.png"
                s.draw_legend()
            else:
                output_file_name += ".png"
        else:
            output_file_name += ".png"

        # Write State space tree
        s.write_image(output_file_name)
    else:
        raise Exception("No solution found")

if __name__ == "__main__":
    main()
```

- def main(): Định nghĩa hàm main.

- `s = Solution()`: Tạo đối tượng `Solution`.
- `if s.solve(solve_method)`: Gọi phương thức `solve`. Nếu tìm thấy giải pháp:
 - Hiển thị giải pháp trên console với `s.show_solution()`.
 - Tạo tên tệp đầu ra dựa trên phương pháp và cờ chú giải.
 - Ghi lại hình ảnh cây trạng thái bằng `s.write_image(outputfile)`.